

Created by → Ananya Tiwari
Registration number: 25BAI11158

Course → Programming in Java

Smart Task Manager - a CLI application

GitHub Repository URL:

<https://github.com/Tetbet/Task-Manager-Project>

1. Introduction

The Smart Task Manager is a lightweight, terminal-based application developed using Core Java. It is designed to help users manage their daily tasks efficiently without the overhead of heavy graphical interfaces. The project applies fundamental Java concepts, including Object-Oriented Programming (OOP), the Collections Framework, Exception Handling, and Multithreading, to deliver a seamless and robust user experience directly from the command line.

2. Problem Statement

In today's fast-paced environment, people often struggle to keep track of their tasks and priorities. Existing GUI task managers can be distracting or consume too many system resources. There is a need for a simple, fast, and terminal-based application that allows users to create tasks, assign priorities, quickly undo mistakes, and receive background reminders without leaving their command-line environment.

3. Functional Requirements

The system provides the following functional capabilities:

- Task Management Module: Users can add standard tasks and view all pending/completed tasks.
- Priority Management Module: Users can create specialized tasks tagged with priority levels (High/Medium/Low).
- Task Tracking: Users can mark specific tasks as completed using their unique Task ID.
- Action History (Undo): Users can undo the most recent task addition if they made a mistake.
- Background Reminder System: A background thread periodically checks for incomplete tasks and alerts the user automatically.

4. Non-functional Requirements

- Usability: Provides an intuitive, numbered, menu-driven CLI for easy navigation.
- Error Handling: Implements robust exception handling so the program doesn't crash if a user inputs a letter instead of a number.
- Performance: Uses lightweight, in-memory data structures (ArrayList, Stack) for instant read/write operations.

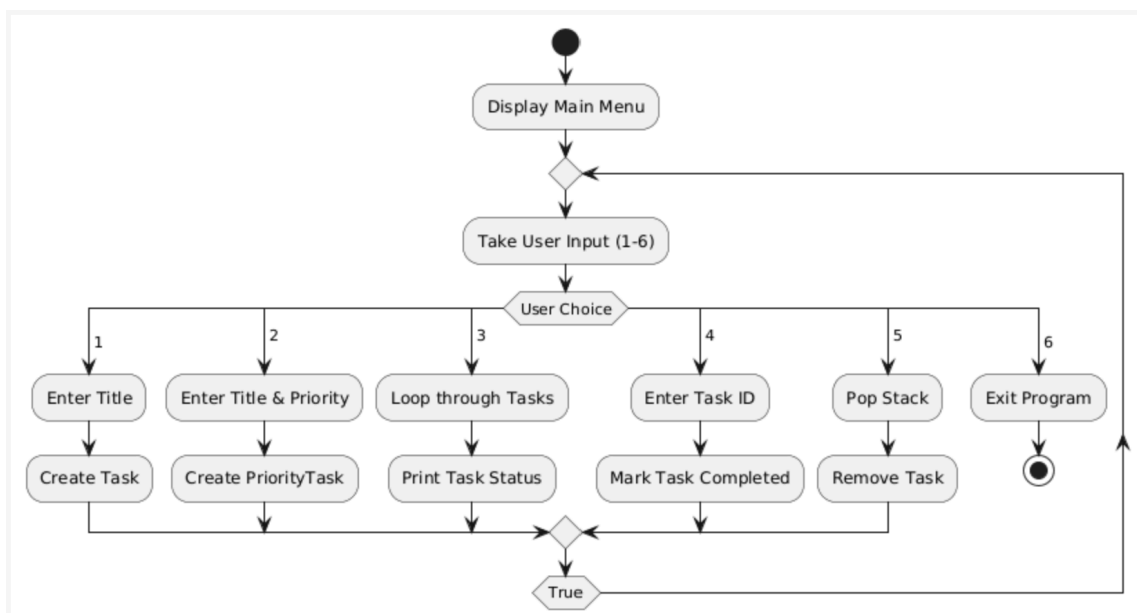
- **Maintainability:** Code is highly modular and utilizes OOP principles, making future updates easy.

5. System Architecture

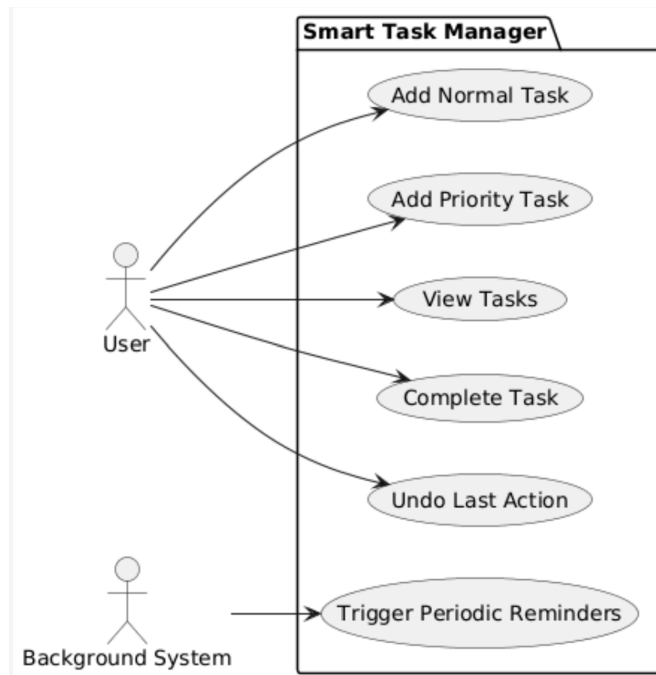
The application uses a monolithic, in-memory architecture. The Main class serves as the entry point, handling the CLI menu loop and user inputs. It delegates business logic to the TaskManager class, which acts as the central controller interacting with Task objects. A separate daemon thread (ReminderThread) runs concurrently in the background, reading the same task list to generate periodic alerts.

6. Design Diagrams

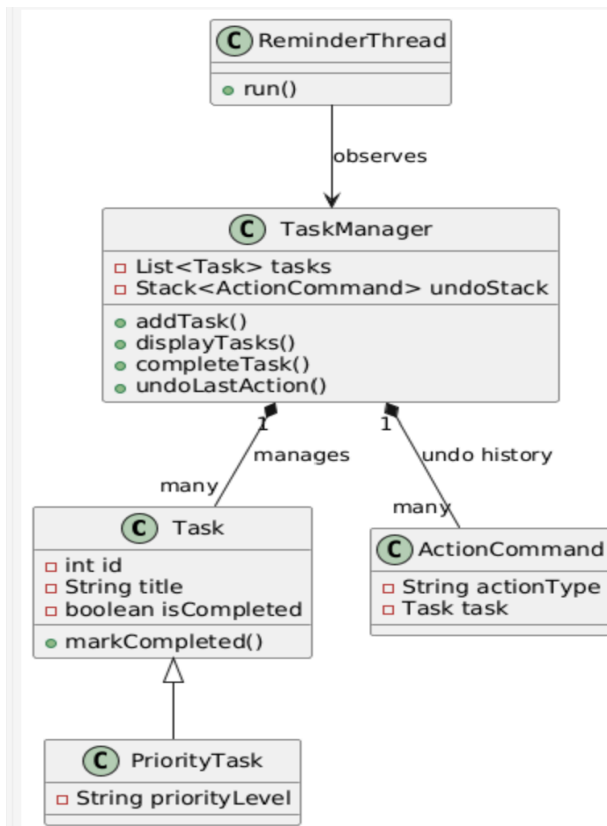
A. Process/Workflow diagram



B. Use Case Diagram



C. Class Diagram



7. Design Decisions & Rationale

- **ArrayList for Storage:** Chose ArrayList for storing tasks because it provides fast, $O(1)$ time complexity for reading tasks when generating lists.
- **Stack for Undo Feature:** The Last-In-First-Out (LIFO) property of a Stack makes it the mathematically perfect data structure for implementing an "Undo" feature.
- **Daemon Thread for Reminders:** Chose to make the `ReminderThread` a daemon thread so that it automatically shuts down when the user exits the main menu, preventing the application from hanging in the background.

8. Implementation Details

The project is implemented entirely in Java.

- **Module 1 (Basics):** Uses `Scanner` for I/O, and switch-case inside an infinite while loop for the menu.

- Module 2 (OOP): `PriorityTask` extends the `Task` class (Inheritance) and overrides the `toString()` method (Polymorphism) to display priority labels.
- Module 3 (Exceptions & Threads): Created custom exceptions (`InvalidInputException`, `TaskNotFoundException`) to handle business logic errors safely. Implemented the `Runnable` interface for concurrent background reminders.
- Module 4 (Collections): Utilized `java.util.List` and `java.util.Stack`.

9. Screenshots / Results

A. Welcome screen

```
=====
Welcome to Smart Task Manager
=====

1. Add Normal Task
2. Add Priority Task
3. View Tasks
4. Complete a Task
5. Undo Last Add
6. Exit
Choose an option: 
```

B. Adding a Task

```
Choose an option: 1
Enter task title: Study Java
Task added successfully!

1. Add Normal Task
2. Add Priority Task
3. View Tasks
4. Complete a Task
5. Undo Last Add
6. Exit
Choose an option:
[BACKGROUND REMINDER]: You have 1 pending task(s) to complete!
Choose an option:
[BACKGROUND REMINDER]: You have 1 pending task(s) to complete!
Choose an option: 
```

C. Completing a task

```
Choose an option: 4
Enter Task ID to complete: 1
Task marked as completed!

1. Add Normal Task
2. Add Priority Task
3. View Tasks
4. Complete a Task
5. Undo Last Add
6. Exit
Choose an option: 
```

10. Testing Approach

Testing was conducted manually via the command line environment:

- **Positive Testing:** Added tasks, viewed them, and marked them as complete to ensure happy-path logic works.
- **Negative Testing:** Entered alphabet characters when the menu asked for an integer to ensure `InputMismatchException` was caught and the app didn't crash. Tried completing a task ID that doesn't exist to test `TaskNotFoundException`.

11. Challenges Faced

- **Scanner Input Bugs:** Encountered an issue where `Scanner.nextInt()` didn't consume the newline character, causing the next `nextLine()` prompt to skip.
- Resolved this by adding an extra `scanner.nextLine()` after integer inputs.

- **Thread Interruption:** Ensuring the background reminder printed out cleanly without interrupting the user's current typing in the command line required careful formatting of print statements.

12. Learnings & Key Takeaways

- Gained practical experience implementing Inheritance and Polymorphism in a real-world scenario.
- Learned how the Java Collections Framework (specifically Stack) simplifies complex features like Action History (Undo).
- Understood how Multithreading works in Java, specifically the difference between standard threads and Daemon threads.

13. Future Enhancements

- **File I/O Integration:** Add functionality to save tasks to a .txt or .csv file so data persists after the application is closed.
- **Sorting Algorithms:** Implement sorting so users can view tasks ordered by Priority Level rather than just creation order.

- Database Integration: Connect the application to a lightweight database like SQLite.

14. References

- Oracle Java Documentation (docs.oracle.com)
- GeeksforGeeks - Java Collections and Multithreading tutorials.
- Course study materials and lecture notes.